

Chapter 3:

Processes Concept

Chien Chin Chen

Department of Information Management
National Taiwan University

Outline

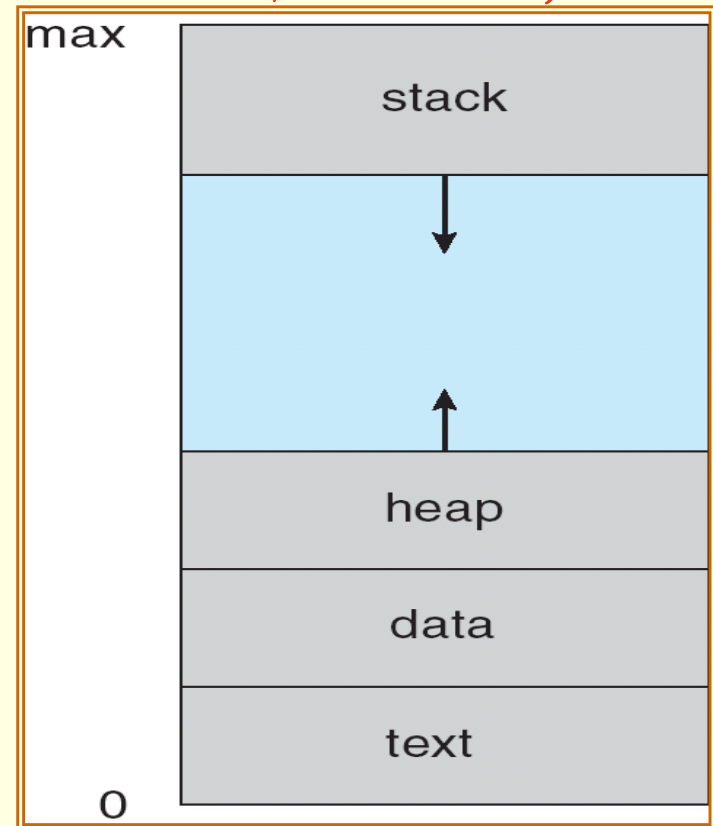
- Overview
- Process Scheduling
- Operations on Processes
- Interprocess Communication
- Examples of IPC Systems
- Communication in Client-Server Systems

Process Concept (1/3)

- The name of CPU activities varies in different systems:
 - Batch system – *jobs*.
 - Time-shared systems – *user programs* or *tasks*.
- Textbook uses the terms ***job*** and ***process*** almost interchangeably, but prefers *process*.
 - However, much of operating-system theory and terminology was developed during the batch era.
 - *Job* (such as job scheduling) would be used to avoid misunderstanding.

Process Concept (2/3)

- Process: *正在執行中的 program, 比 program 多了 (program 只有 code)*
 - A program in execution.
 - Execution must progress in **sequential** fashion.
- A process in memory includes:
 - **Text section**: the program code.
 - **Stack**: contains temporary data (local variable, function parameters ...).
 - **Data section**: contains **global** variables.
 - **Heap**: used for dynamical memory allocation.
 - **Program counter**: the address of next instruction.
 - **Register status**.



Process in memory

Process Concept (3/3)

- A program is not a process:
 - A program is a **passive** entity; a list of instruction stored on disk.
 - A process is an **active** entry.
 - A program becomes a process when an executable file is loaded into memory.
- Two (or more) processes may be associated with the same program.
 - For example, a user may invoke many copies of the web browser program.
 - Each of these is a separate process.
 - **While the text sections are equivalent, the data, heap, and stack sections vary.**

Process State (1/2)

- The **state** of a process is defined by the current activity of that process.
- Each process may be in one of the following states:
 - **New**: the process is being created. 在硬碟上有但尚未載入記憶體時
 - **Running**: instructions are being executed.
 - **Waiting**: the process is waiting for some event to occur.
 - Such as an I/O completion. 變數還沒到手, 還不能給CPU跑
 - **Ready**: the process is waiting to be assigned to a processor. 有資格使用CPU, 被挑到會變成running
 - **Terminated**: the process has finished execution.
- **Only one process can be running on any processor at any instance.**
 - Many processes may be *ready* and *waiting*.

Process State (2/2)

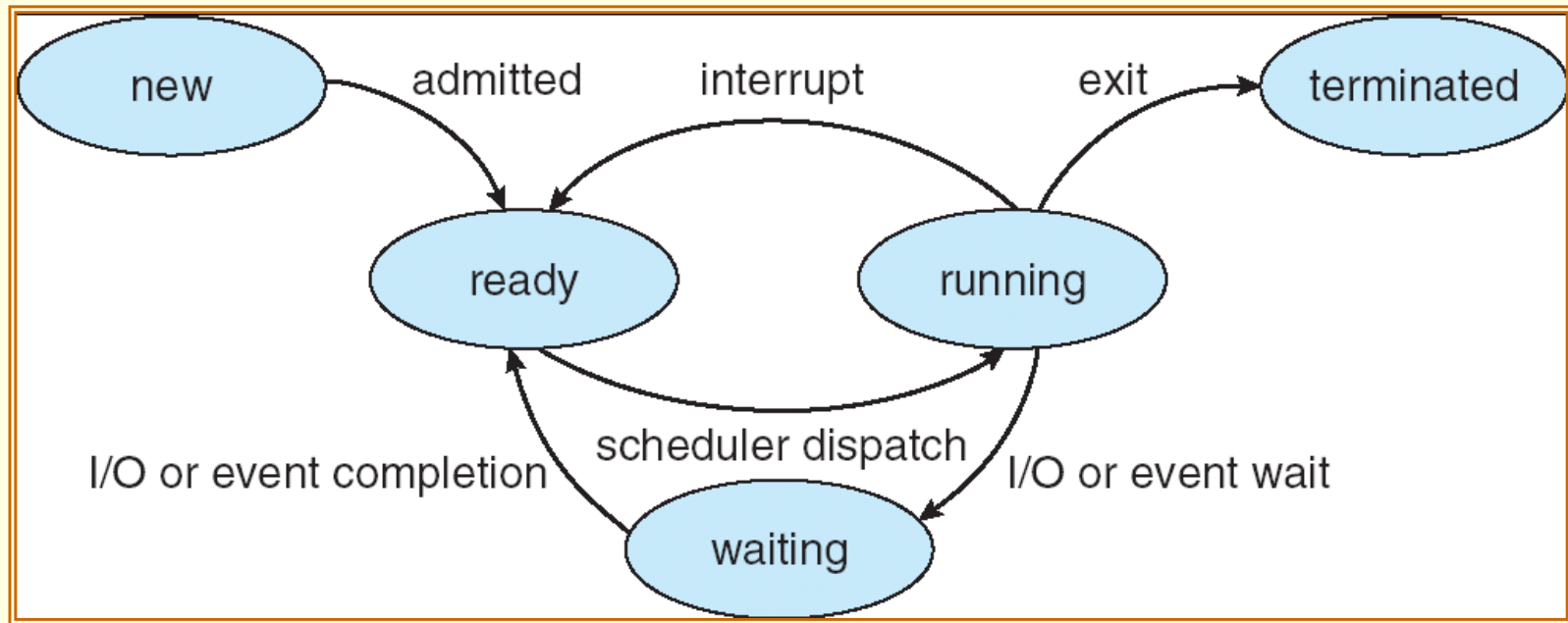
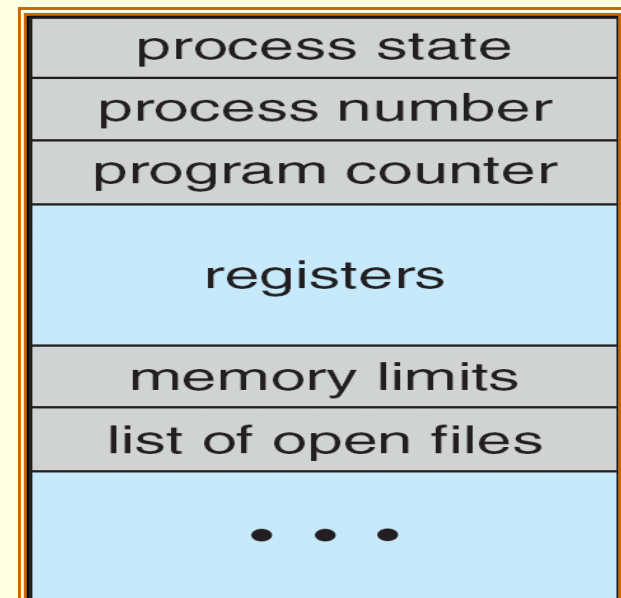


Diagram of Process State

Process Control Block

- **Process control block** (PCB): a representation of a process in the operating system.
- Information associated with each PCB:
 - **Process state.**
 - **Program counter.**
 - The address of next instruction.
 - **CPU registers.**
 - **CPU scheduling information.**
 - Process priority (chapter 5).
 - **Memory-management information** (chapter 8).
 - **Accounting information.**
 - E.g., amount of CPU time.
 - **I/O status information.**
 - Allocated devices, files, and so on.



Different processes have different PCB information.

Process Scheduling

- Multiprogramming:
 - To have some process running at all time.
 - To maximize CPU utilization.
- Time sharing:
 - To switch the CPU among processes so frequently.
 - Users can interact with each program while it is running.
- To do so ... we need ...
 - ***Process scheduler:***
 - Select an available process for execution on the CPU.
 - The rest have to wait until the CPU is free and can be rescheduled.

Scheduling Queues and Schedulers

(1/7)

- The operating system usually contains a lot of **queues**.
 - Contain pointers to the first and final PCBs in the list (queue).
 - Each PCB includes a pointer field that points to the next PCB in the queue.
 - **Job queue:** (舊時代的)
 - Processes are initially spooled to a mass-storage device (e.g., a disk), where they are kept for later execution.
 - Job scheduler later selects processes from this pool and load them into memory for execution.
 - Often in batch system. **On some operating systems, job queue and job scheduler may be absent.**
 - Such as UNIX and MS Windows.
 - Just put every new process in memory for execution.

Scheduling Queues and Schedulers

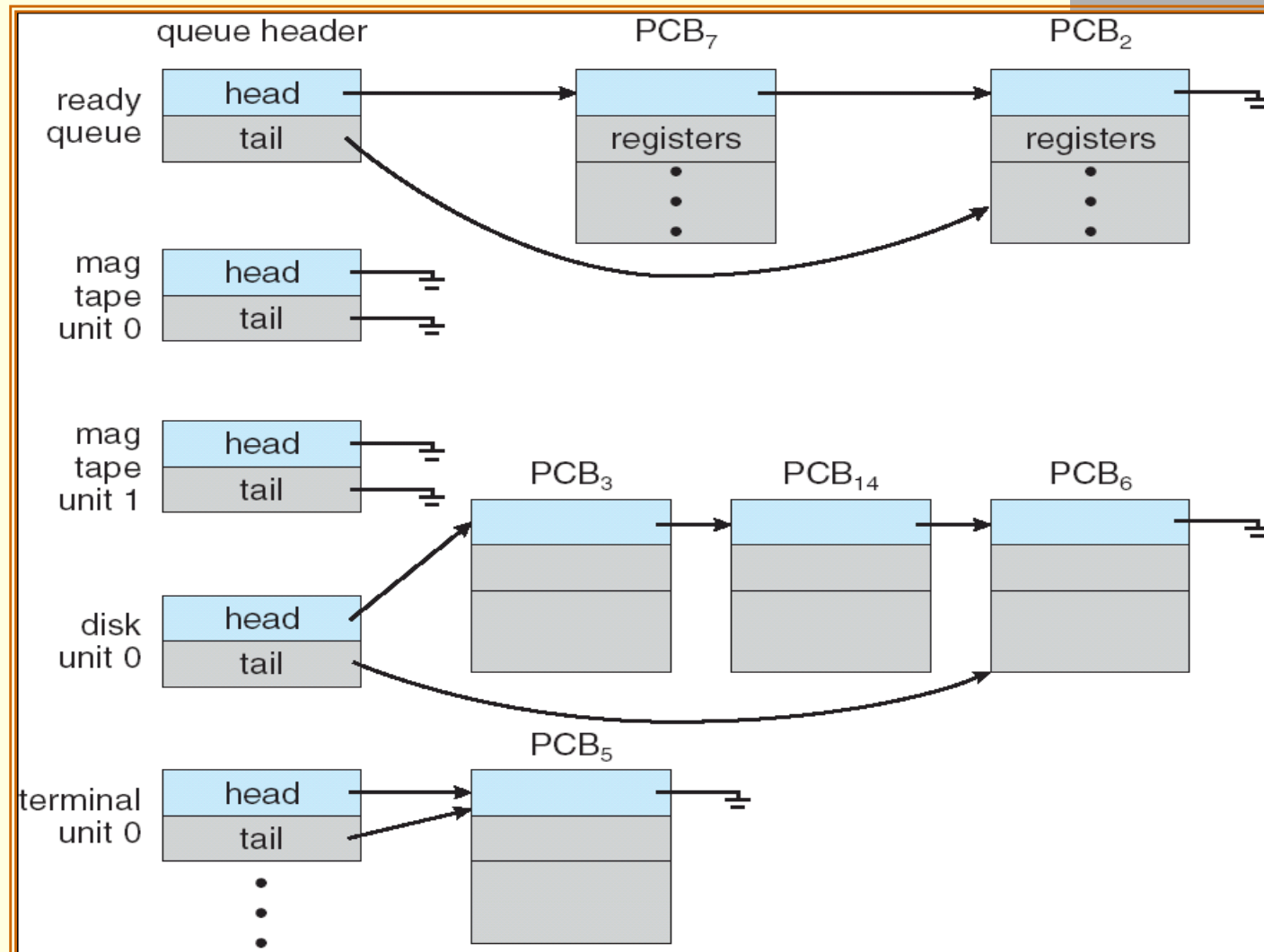
(2/7)

- **Ready queue:**
 - Contain the processes residing in main memory.
 - Those processes are **ready** and waiting to execute.

- Operating systems also include other queues.
 - **Device queue:**
 - A list of processes waiting for a particular I/O device.
 - For instance, there are many processes make requests to a disk.

Scheduling Queues and Schedulers

(3/7)



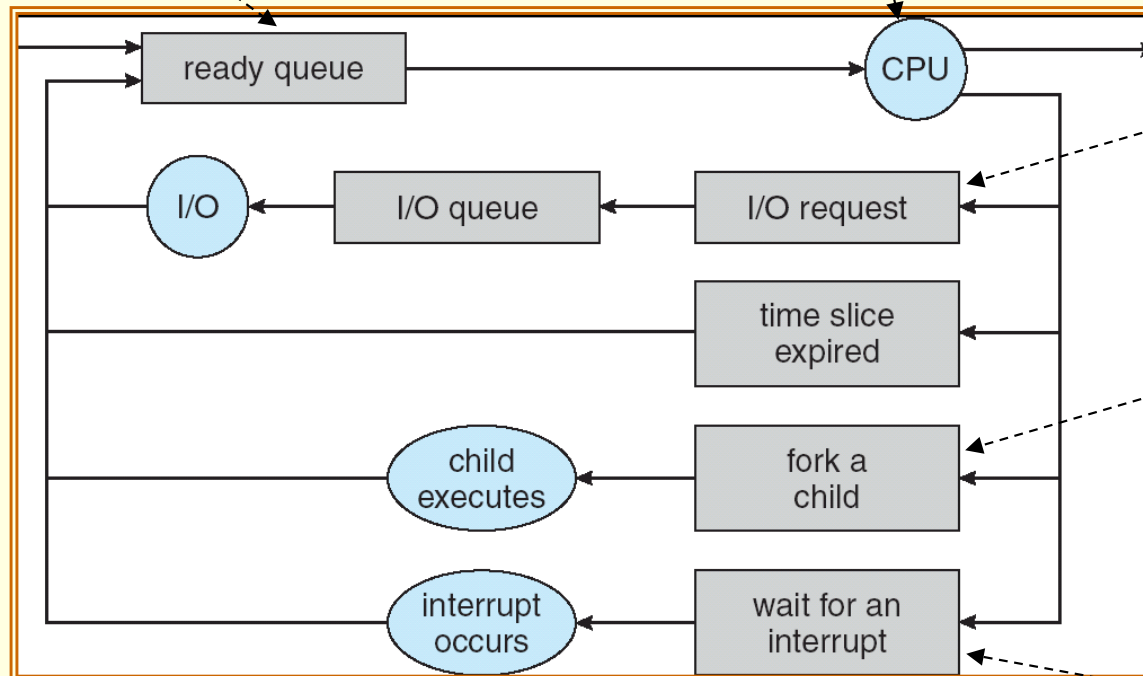
Ready Queue And Various I/O Device Queues

Scheduling Queues and Schedulers

(4/7)

A new process is initially put in the ready queue (**ready** state).

Eventually, it will be selected (dispatched) for execution (**running** state).



The process may issue an I/O request, and switches to the **waiting** state.

The process may create a new sub-process and wait for the sub-process's termination (**waiting** state).

The process may be removed from the CPU by an interrupt, and be put back in the ready queue (**ready** state).

Processes migrate among the various queues.

When the process terminates, it is removed from all Queues and has its PCB and resources deallocated.

Scheduling Queues and Schedulers

(5/7)

- A process migrates among the various scheduling queues throughout its lifetime.
 - The operating system must select process from these queues in some fashion.
- Two main schedulers:
 - **Job scheduler** (or **long-term** scheduler) – selects which processes should be brought into the ready queue.
 - **CPU scheduler** (or **short-term** scheduler) – selects which process should be executed next and allocates CPU.
- The primary difference between the schedulers: **frequency of execution**.
 - Often, the short-term scheduler executes at least once every 100 ms.
 - The long-term scheduler executes much less frequently.

Scheduling Queues and Schedulers

(6/7)

- The short-term scheduler must be **fast**.
 - If it takes 10 ms to decide to execute a process for 100 ms, then $10 / (10 + 100) = 9\%$ of the CPU is being wasted for scheduling the work.
- However, because of the longer interval between executions, the long-term scheduler can afford to take more time to decision.
- The long-term scheduler controls the degree of multiprogramming.
- In general, processes can be described as either **I/O bound** or **CPU bound**.
 - A **I/O-bound process** spends more of its time doing I/O.
 - A **CPU-bound process** uses more of its time doing computations.
 - A good long-term scheduler should select a good process **mix** of I/O-bound and CPU-bound processes that the system will be balanced.

Scheduling Queues and Schedulers

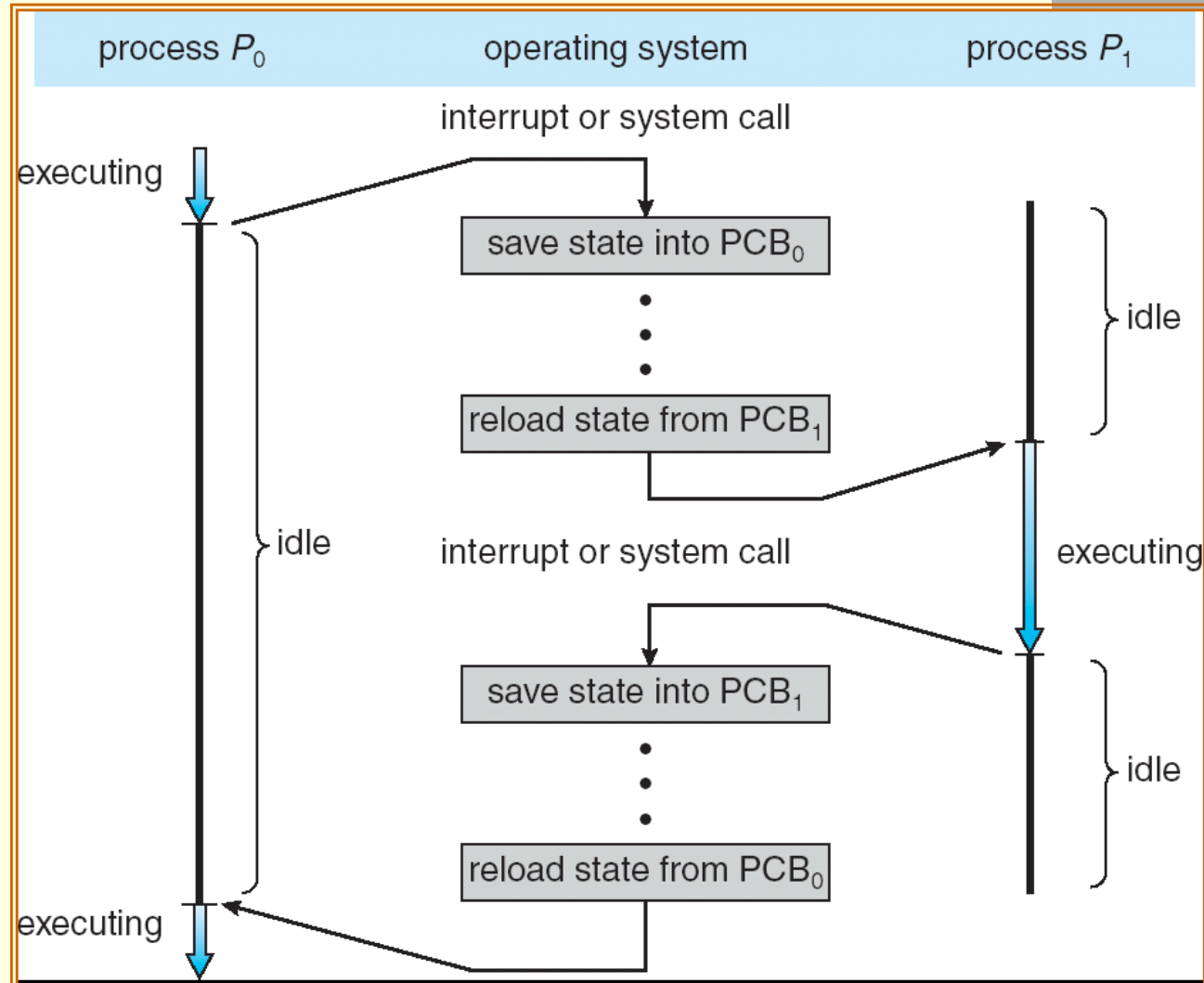
(7/7)

- For systems with no long-term schedulers:
 - The stability of the systems may depend on the self-adjusting nature of human users.
 - If the performance declines to unacceptable levels on a multi-users system, some users will simply quit.

Context Switch (1/3)

- When CPU switches to another process, the system must **save** the **context** of the old process and **load** the saved **context** for the new process.
 - E.g., interrupts cause the operating system to change a CPU from its current task to run a kernel routine.
- Such a switching is known as a ***context switch***.
- The context is represented in the PCB of the process.
 - The value of the CPU registers.
 - The process state.
 - Program counter.
 - ...

Context Switch (2/3)



Context Switch (3/3)

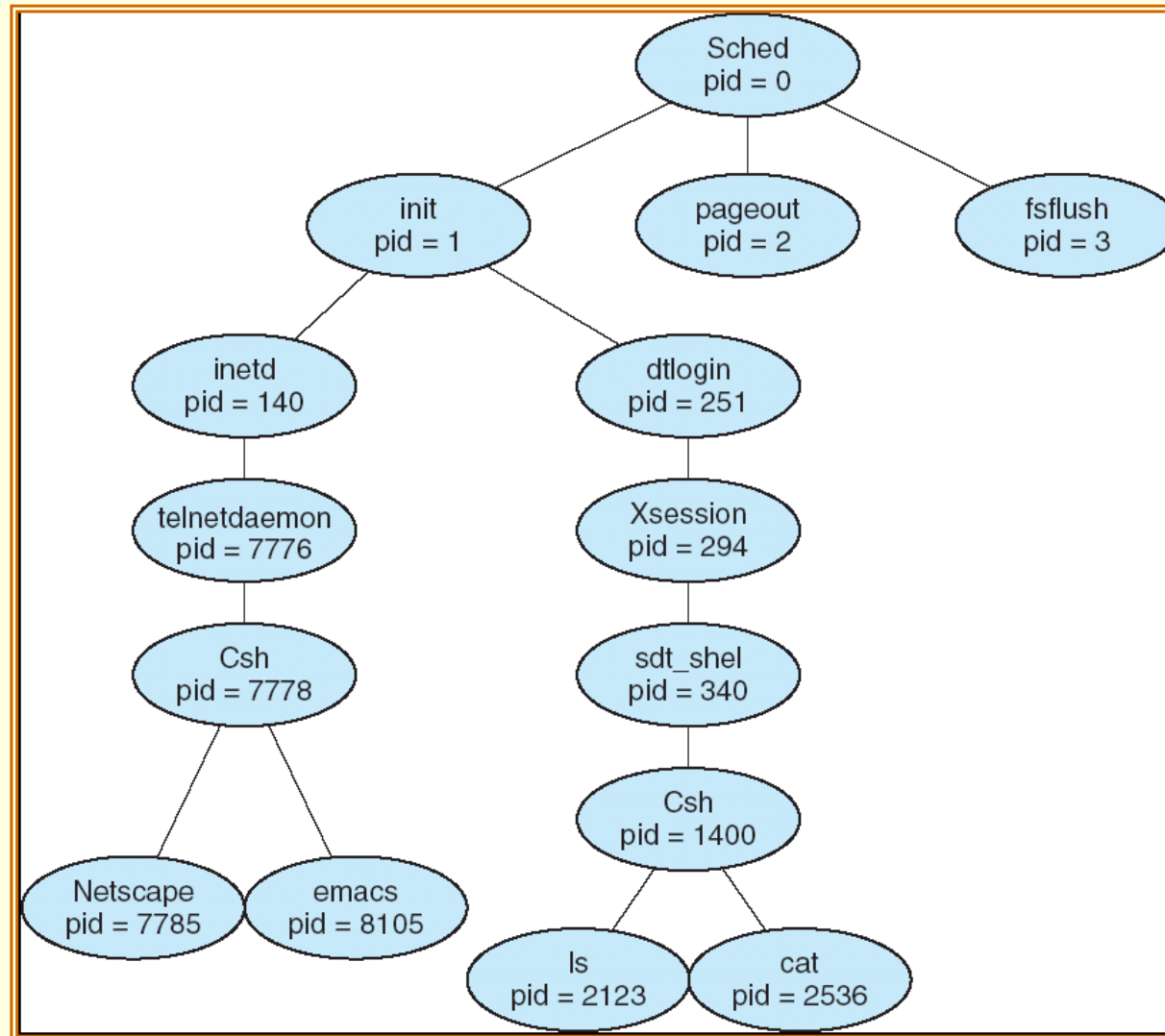
- **Context-switch time is overhead**; the system does no useful work while switching.
- Dependent on hardware support.
 - Some processors provide multiple sets of registers.
 - A context switch simply requires changing the pointer to the current register set.

Operations on Processes

Process Creation (1/8)

- A process may create several new processes, via a create-process system call.
- The creating process is called a **parent** process, and the new processes are called the **children** of that process.
 - New processes may in turn create other processes, forming a **tree** of processes.
- Most operating systems (UNIX and MS Windows) identify processes according to a unique process identifier (pid).
 - Usually an integer number.

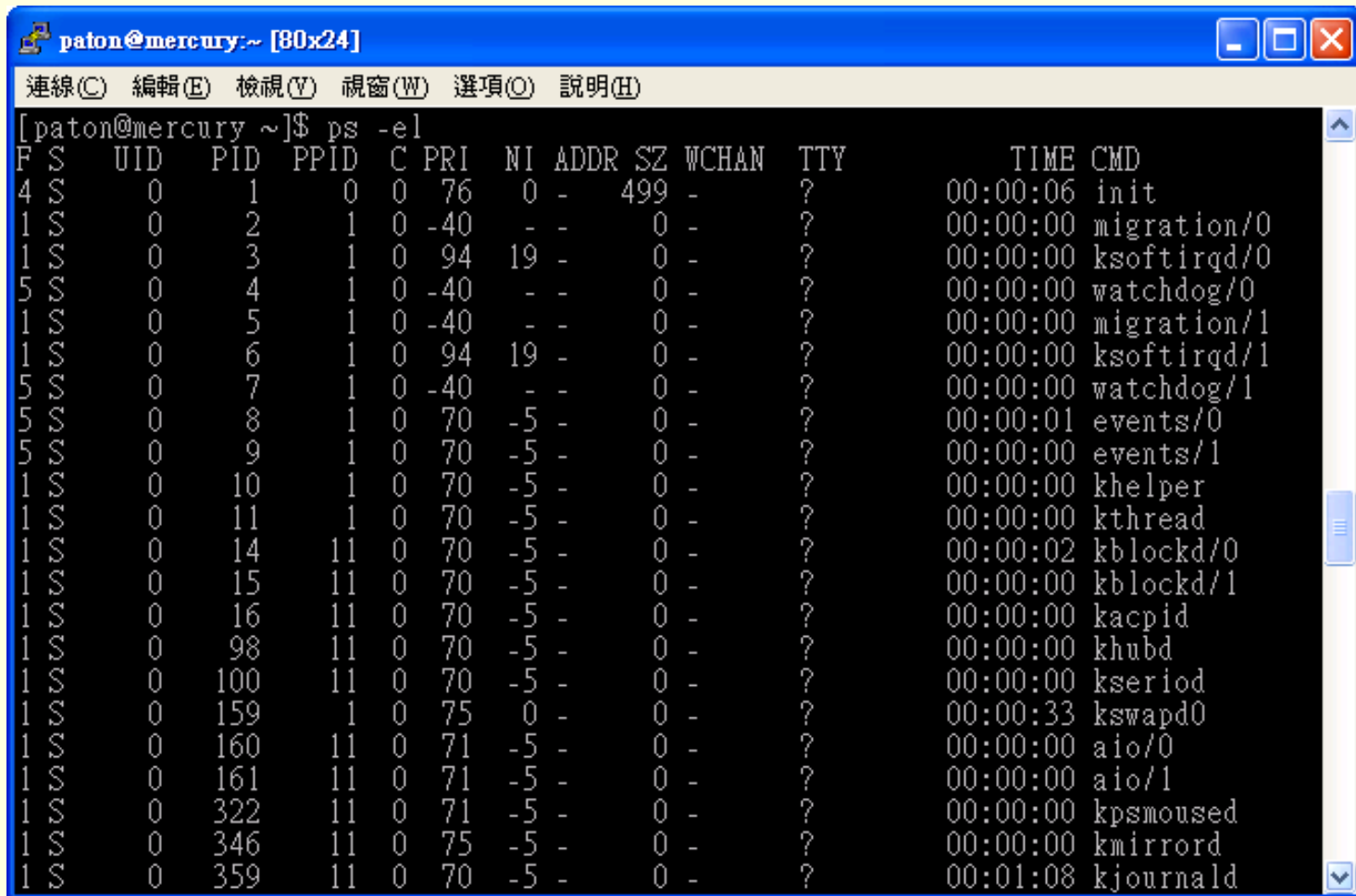
Process Creation (2/8)



A tree of processes on a typical Solaris

Process Creation (3/8)

- On UNIX, the `ps` command can be used to obtain the information of all process.



A terminal window titled "paton@mercury:~ [80x24]" showing the output of the command `ps -el`. The window has a menu bar with options: 連線(C), 編輯(E), 檢視(V), 視窗(W), 選項(O), 說明(H). The output is a table of process information.

F	S	UID	PID	PPID	C	PRI	NI	ADDR	SZ	WCHAN	TTY	TIME	CMD
4	S	0	1	0	0	76	0	-	499	-	?	00:00:06	init
1	S	0	2	1	0	-40	-	-	0	-	?	00:00:00	migration/0
1	S	0	3	1	0	94	19	-	0	-	?	00:00:00	ksoftirqd/0
5	S	0	4	1	0	-40	-	-	0	-	?	00:00:00	watchdog/0
1	S	0	5	1	0	-40	-	-	0	-	?	00:00:00	migration/1
1	S	0	6	1	0	94	19	-	0	-	?	00:00:00	ksoftirqd/1
5	S	0	7	1	0	-40	-	-	0	-	?	00:00:00	watchdog/1
5	S	0	8	1	0	70	-5	-	0	-	?	00:00:01	events/0
5	S	0	9	1	0	70	-5	-	0	-	?	00:00:00	events/1
1	S	0	10	1	0	70	-5	-	0	-	?	00:00:00	khelper
1	S	0	11	1	0	70	-5	-	0	-	?	00:00:00	kthread
1	S	0	14	11	0	70	-5	-	0	-	?	00:00:02	kblockd/0
1	S	0	15	11	0	70	-5	-	0	-	?	00:00:00	kblockd/1
1	S	0	16	11	0	70	-5	-	0	-	?	00:00:00	kacpid
1	S	0	98	11	0	70	-5	-	0	-	?	00:00:00	khubd
1	S	0	100	11	0	70	-5	-	0	-	?	00:00:00	kseriod
1	S	0	159	1	0	75	0	-	0	-	?	00:00:33	kswapd0
1	S	0	160	11	0	71	-5	-	0	-	?	00:00:00	aio/0
1	S	0	161	11	0	71	-5	-	0	-	?	00:00:00	aio/1
1	S	0	322	11	0	71	-5	-	0	-	?	00:00:00	kpsmoused
1	S	0	346	11	0	75	-5	-	0	-	?	00:00:00	kmirrord
1	S	0	359	11	0	70	-5	-	0	-	?	00:01:08	kjournald

Process Creation (4/8)

- When parent process holds certain resources (files, I/O devices).
 - Parent and children share all resources.
 - Children share subset of parent's resources.
 - Parent and child share no resources.
- When a process creates a new process, two possibilities exist in terms of execution:
 - Parent and children execute concurrently.
 - Parent waits until children terminate.
- There are also two possibilities in terms of the address space of the new process:
 - Child duplicate of parent.
 - Child has **a program loaded into it.**

Process Creation (5/8)

- UNIX examples:

- fork() system call creates new process (POSIX).
- The new process consists of **a copy of the address space of the original process**.
- Both process **continue (concurrently) execution** at the instruction after the `fork()`.
- But ... how to distinguish?
 - The return code for the `fork()` is zero for the new (child) process.
 - The process identifier (PID) of the child is returned to the parent.

Process Creation (6/8)

```
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    pid_t pid;
```

→ - 一种 int.

```
    pid = fork(); /* fork another process */
```

```
    if (pid < 0) { /* error occurred */
```

```
        fprintf(stderr, "Fork Failed\n");
```

```
        return 1; // exit(1);
```

```
    }
```

```
    else if (pid == 0) { /* child process */
```

```
        execlp("/bin/ls", "ls", NULL);
```

```
    }
```

```
    else { /* parent process */
```

```
        /* parent will wait for the child to complete */
```

```
        wait(NULL);
```

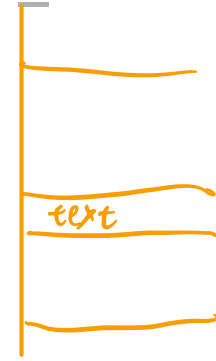
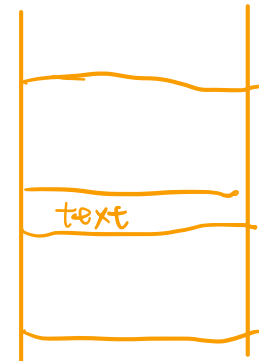
```
        printf ("Child Complete\n");
```

```
    }
```

```
    return 0; // exit(0);
```

```
}
```

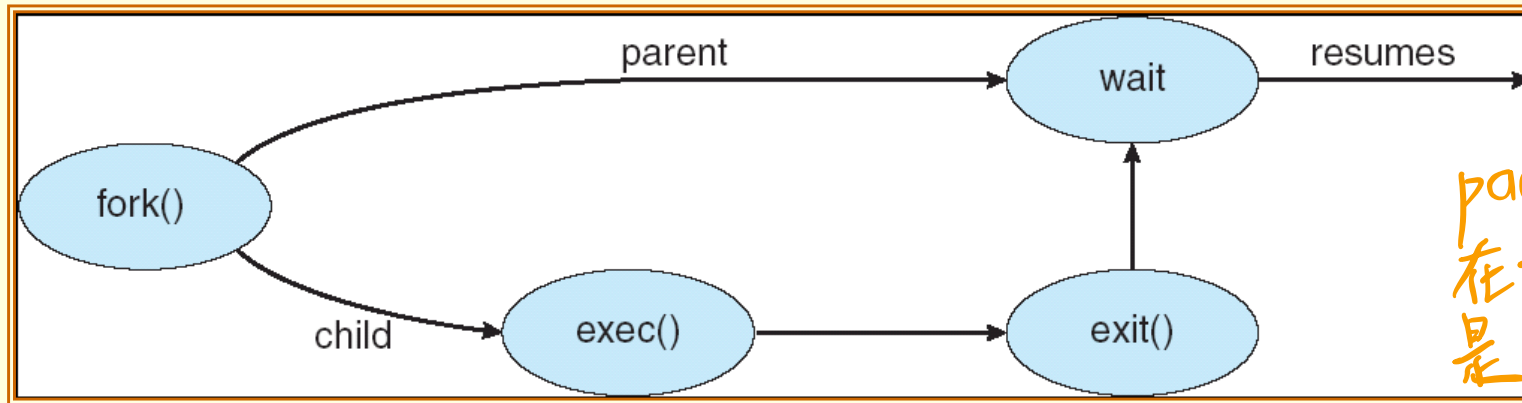
fork_test.c



Process Creation (7/8)

- Typically, the **exec()** (or its family, such as **execvp()**) system call (POSIX) is used after a `fork()` by one of the two processes to **replace the process's memory space with a new program**.
 - For example, the child process overlays its address space with UNIX command `/bin/ls` using the `execvp()` system call.
- The parent can wait for the child process to complete with the **wait()** system call (POSIX).
 - When the child process completes, the parent process resumes from the call to `wait`.
- **exit()** system call: cause normal process termination (POSIX).
 - Or indirectly called by a `return` statement in `main()`.

Process Creation (8/8)



parent & child
在 fork 當下都
是 ready

↓

parent 會 wait
child 跑完才
能回到 ready

```
For OS Course [執行中] - Oracle VM VirtualBox
檔案 機器 檢視 輸入 裝置 說明
paton@paton-VirtualBox: ~/OS_exercises/CH03
paton@paton-VirtualBox:~/OS_exercises/CH03$ gcc -o fork_test.out fork_test.c
paton@paton-VirtualBox:~/OS_exercises/CH03$ ./fork_test.out
總計 72
drwxrwxr-x 2 paton paton 4096 3月 14 10:38 .
drwxrwxr-x 3 paton paton 4096 3月 13 19:28 ..
-rw-rw-r-- 1 paton paton 394 3月 13 19:38 consumer.c
-rwxrwxr-x 1 paton paton 8768 3月 13 19:47 consumer.out
-rw-rw-r-- 1 paton paton 499 3月 14 10:38 fork_test.c
-rwxrwxr-x 1 paton paton 8856 3月 14 10:38 fork_test.out
-rw-rw-r-- 1 paton paton 885 3月 13 19:51 pipe.c
-rwxrwxr-x 1 paton paton 9096 3月 13 19:51 pipe.out
-rw-rw-r-- 1 paton paton 806 3月 13 19:37 producer.c
-rwxrwxr-x 1 paton paton 8816 3月 13 19:47 producer.out
Child Complete
paton@paton-VirtualBox:~/OS_exercises/CH03$
```

Process Termination

- Process executes last statement and asks the operating system to delete it by using the `exit()` system call.
 - Can return a status value (typically an integer) to its parent process (via the `wait()` system call).
 - Process' resources are deallocated by operating system.
- Parent may terminate execution of children processes (`kill()`, **POSIX**) when: *parent 可以主動殺 child process*
 - Child has exceeded allocated resources.
 - Task assigned to child is no longer required.
 - If parent is exiting.
 - Some operating systems do not allow child to continue if its parent terminates.
 - All children terminated - **cascading termination**.



Interprocess Communication

Cooperating Processes (1/4)

- Processes may be **independent** processes or **cooperating** processes.
 - **Independent** process cannot affect or be affected by the execution of another process.
 - **Cooperating** process can affect or be affected by the execution of another process.

- Reasons for process cooperation:
 - Information sharing.
 - Computation speed-up.
 - Can be achieved only if the system have multiple CPUs.
 - Modularity.
 - Convenience.
 - Users may work on many tasks (or sub-tasks) at the same time.

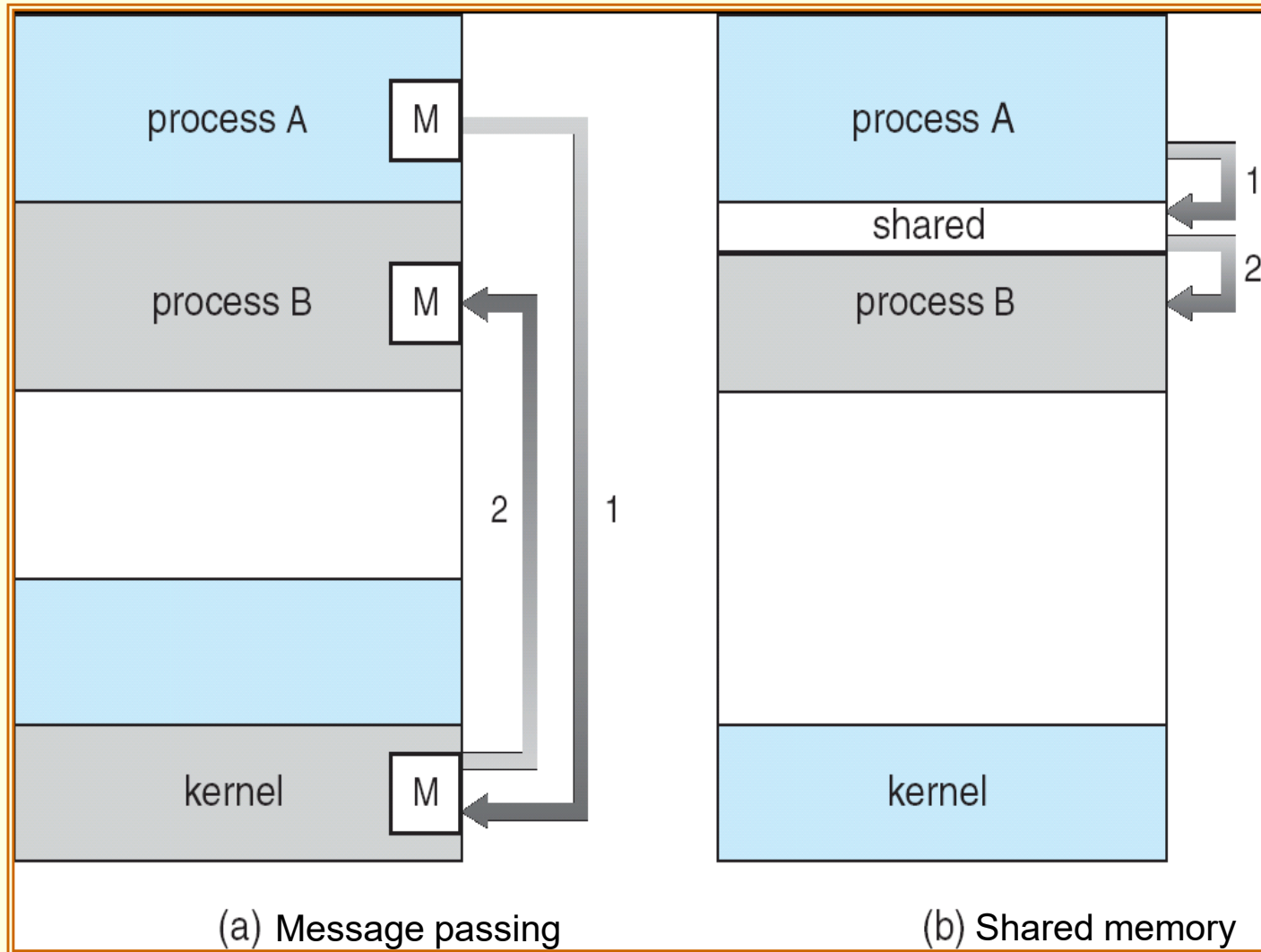
Cooperating Processes (2/4)

- **Interprocess communication** (IPC) is a mechanism that allows cooperating processes to exchange data and information.
- Two typical models of IPC:
 - **Shared memory.**
 - A region of memory is shared by cooperating processes.
 - Processes can exchange information by reading and writing data to the region.
 - **Message passing.**
 - Communication takes place by means of messages exchanged between the processes.
- Both of the models are common and implemented in many operating systems.

Cooperating Processes (3/4)

- Message passing is useful for exchanging smaller amounts of data.
 - Because **no conflicts** need be avoided.
 - Usually **slow**, because message-passing systems are generally implemented as system calls.
 - Require the more time-consuming task of kernel intervention.
- Shared memory allows maximum speed and convenience of communication.
 - It can be done at memory speeds within a computer.
 - Is **faster** than message passing.
 - Once shared memory is established, all accesses are treated as routine memory access.
 - No assistance from the kernel is required.

Cooperating Processes (4/4)



Shared-Memory Systems (1/4)

- A process creates a shared-memory region residing in its address space.
- Other processes that wish to communicate using this shared-memory segment must attach it to their address space.
- Then, they can exchange information by reading and writing data in the shared area.
- The processes are responsible for ensuring that they are not writing to the same location simultaneously.
 - Chapter 6 will discuss synchronization to synchronize concurrent accesses.

Shared-Memory Systems (2/4)

- Shared memory as a **producer-consumer problem**:
 - **producer** process produces information that is consumed by a **consumer** process.
 - A **buffer** (shared memory) can be filled by the producer and emptied by the consumer.
- Two types of buffers:
 - unbounded-buffer places no practical limit on the size of the buffer. 無容量限制
 - The consumer may have to wait for new items.
 - But the producer can always produce new items.
 - bounded-buffer assumes that there is a fixed buffer size. 實務上常見
 - The consumer may have to wait for new items.
 - The producer have to wait if the buffer is full.

Shared-Memory Systems (3/4)

■ Implementation:

```
#define BUFFER_SIZE 10
typedef struct {
    . . .
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

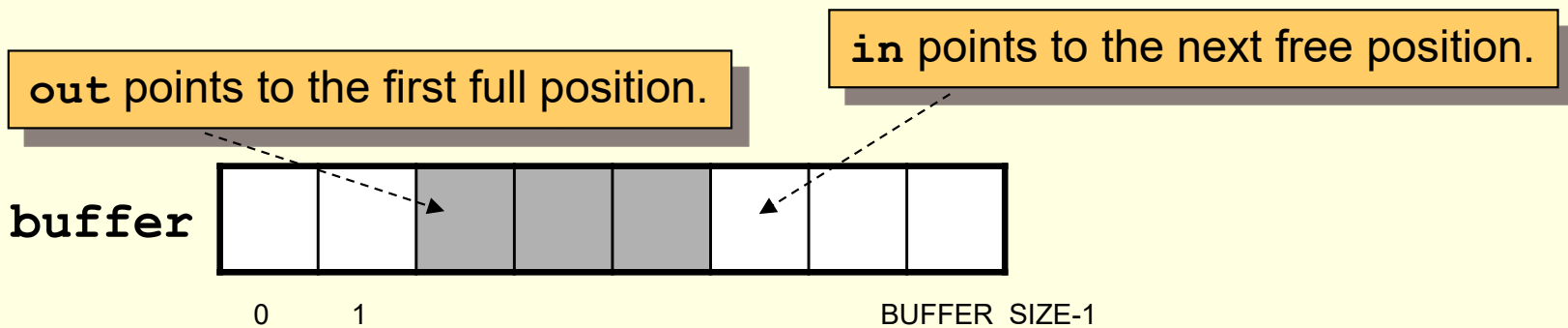
■ The buffer is implemented as a **circular array**:

- `in=(in++)%BUFFER_SIZE`
- `out=(out++)%BUFFER_SIZE`

■ At most `BUFFER_SIZE-1` items in the buffer at the same time.

■ Empty: `in == out`

■ Full: `((in+1)%BUFFER_SIZE) == out`



Shared-Memory Systems (4/4)

■ The producer process:

```
item nextProduced;

while (true) {
    /* Produce an item */
    while (((in + 1) % BUFFER_SIZE) == out)
        ;    // do nothing
    buffer[in] = nextProduced;
    in = (in + 1) % BUFFER_SIZE;
}
```

■ The consumer process

```
item nextConsumed

while (true) {
    while (in == out)
        ;    /* do nothing
    nextConsumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
}
```

→ 一把鎖, $in \neq out$ 後才能通過

POSIX Shared Memory Example (1/7)

- POSIX shared memory is organized using **memory-mapped files**.
 - Associate the region of shared memory with a **file**.
- A process **create** a shared memory object using the `shm_open()` system call.

■ Example:

```
segment_fd = shm_open(name,  
O_CREAT | O_RDWR, 0666);
```

integer file descriptor
for the shared-memory
object

create if it does not yet exist

now, open for reading and writing

access permission

The name of shared-memory object

同组其他人
others

POSIX Shared Memory Example (2/7)

- Then, the `ftruncate()` is used to configure the size of the shared-memory object (in bytes).

- Example:

```
ftruncate(shm_fd, 4096);
```



set the size to 4096 bytes

POSIX Shared Memory Example (3/7)

- Finally, the function `mmap()` map the memory-mapped file into memory.

- Example:

```
ptr = mmap(0, SIZE, PROT_WRITE, MAP_SHARED,  
           shm_fd, 0);
```

starting address of mapping,
NULL(0), then kernel decides the mapping

be written, or PROT_READ be read

offset

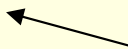
changes to the shared-memory
object will be visible to all
processes sharing the object.

POSIX Shared Memory Example (4/7)

- Finally, a process invokes `shm_unlink()` system call which removes the shared-memory segment.

- Example:

```
shm_unlink(name);
```



The name of shared-memory object

POSIX Shared Memory Example (5/7)

producer.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/mman.h>
#include <sys/types.h>
#include <unistd.h>

int main()
{
    const int SIZE = 4096;
    const char *name = "OS";
    const char *message_0 = "Hello";
    const char *message_1 = "World!";

    int shm_fd;          // shared memory file descriptor
    void *ptr;           // pointer to shared memory object

    shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);

    ftruncate(shm_fd, SIZE);

    ptr = mmap(0, SIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0);

    sprintf(ptr, "%s", message_0);
    ptr += strlen(message_0);
    sprintf(ptr, "%s", message_1);
    ptr += strlen(message_1);

    return 0;
}
```

POSIX Shared Memory Example (6/7)

consumer.c

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/mman.h>

int main()
{
    const int SIZE = 4096;
    const char *name = "OS";
    int shm_fd;
    void *ptr;

    shm_fd = shm_open(name, O_RDONLY, 0666);
    ptr = mmap(0, SIZE, PROT_READ, MAP_SHARED, shm_fd, 0);

    printf("%s\n", (char *) ptr);
    shm_unlink(name);

    return 0;
}
```

POSIX Shared Memory Example (7/7)

For OS Course [執行中] - Oracle VM VirtualBox

檔案 機器 檢視 輸入 裝置 說明

paton@paton-VirtualBox: ~/OS_exercises/CH03

```
paton@paton-VirtualBox:~/OS_exercises/CH03$ gcc -o producer.out producer.c -pthread -lrt
paton@paton-VirtualBox:~/OS_exercises/CH03$ gcc -o consumer.out consumer.c -pthread -lrt
paton@paton-VirtualBox:~/OS_exercises/CH03$ ./producer.out
paton@paton-VirtualBox:~/OS_exercises/CH03$ ls /dev/shm/
OS                                pulse-shm-2246340057  pulse-shm-3132231717  pulse-shm-3648555697  pulse-shm-4083207406
pulse-shm-2022300530  pulse-shm-2553655458  pulse-shm-3398482306  pulse-shm-3679606641
paton@paton-VirtualBox:~/OS_exercises/CH03$ ./consumer.out
HelloWorld!
paton@paton-VirtualBox:~/OS_exercises/CH03$
```

Message-Passing Systems (1/7)

- Processes communicate with each other **without** resorting to **shared variables**.
- The communication actions are synchronized.
- Useful in a distributed (network) environment.
- A message-passing facility provides at least two operations:
 - *send(message)*
 - *receive(message)*
 - message size can be **fixed** or **variable**.
 - **fixed-sized messages** are easy to implement, but makes the programming task difficult.
 - **Variable-sized messages** require a more complex system-level implementation, but the programming task becomes simpler.

Message-Passing Systems (2/7)

- Processes that want to communicate must have a way to refer to each other.
- ***Direct-symmetry-communication:***
 - processes that want to communicate must explicitly name the recipient or sender of the communication.
 - `send(P, message)` – send a message to process P.
 - `receive(Q, message)` – receive a message from process Q.
- ***Direct-asymmetry-communication:***
 - Only the sender names the recipient.
 - `send(P, message)` – send a message to process P.
 - `receive(id, message)` – receive a message from **any** process; the variable `id` is set to the name of the sender.

Message-Passing Systems (3/7)

- The disadvantage in both of these *hard-coding* schemes (symmetric and asymmetric).
 - Changing the identifier of a process may necessitate examining all other processes.
 - All references to the old identifier must be modified to the new identifier.
- *Indirect communication:*
 - The messages are sent to and received from *mailboxes*, or *ports*.
 - Each mailbox has a unique identification.
 - A process can communicate with some other process via a number of different mailboxes.
 - A mailbox may be owned either by a process or by the operating system.
 - `send(A, message)` – send a message to mailbox A.
 - `receive(A, message)` – receive a message from mailbox A.

Message-Passing Systems (4/7)

- In indirect communication scheme, a communication link has the following properties:
 - A link is established between a pair of processes only if both members of the pair have a shared mailbox.
 - A link may associated with more than two processes.
 - Between each pair of communicating processes, there may be a number of different links.

Message-Passing Systems (5/7)

- A practical problem of mailbox sharing:
 - P_1 , P_2 , and P_3 share mailbox A.
 - P_1 sends; P_2 and P_3 receive.
 - Who gets the message?

- Solutions
 - Allow a link to be associated with at most two processes.
 - Allow only one process at a time to execute a receive operation.
 - Allow the system to select arbitrarily the receiver. Sender is notified who the receiver was.

Message-Passing Systems (6/7)

- There are different design options for implementing the `send()` and `receive()` primitives.
 - **Blocking** (*synchronous*) **send**: the sending process is blocked until the message is received by the receiving process or by the mail box.
 - **Nonblocking** (*asynchronous*) **send**: the sending process sends the message and resumes operation.
 - **Blocking receive**: the receiver blocks until a message is available.
 - **Nonblocking receive**: the receiver retrieves either a valid message or a null.
- Different combinations of `send()` and `receive()` are possible.
 - E.g., **rendezvous**: `send()` and `receive()` are blocking, a trivial solution to the producer-consumer problem.

Message-Passing Systems (7/7)

■ **Buffering:**

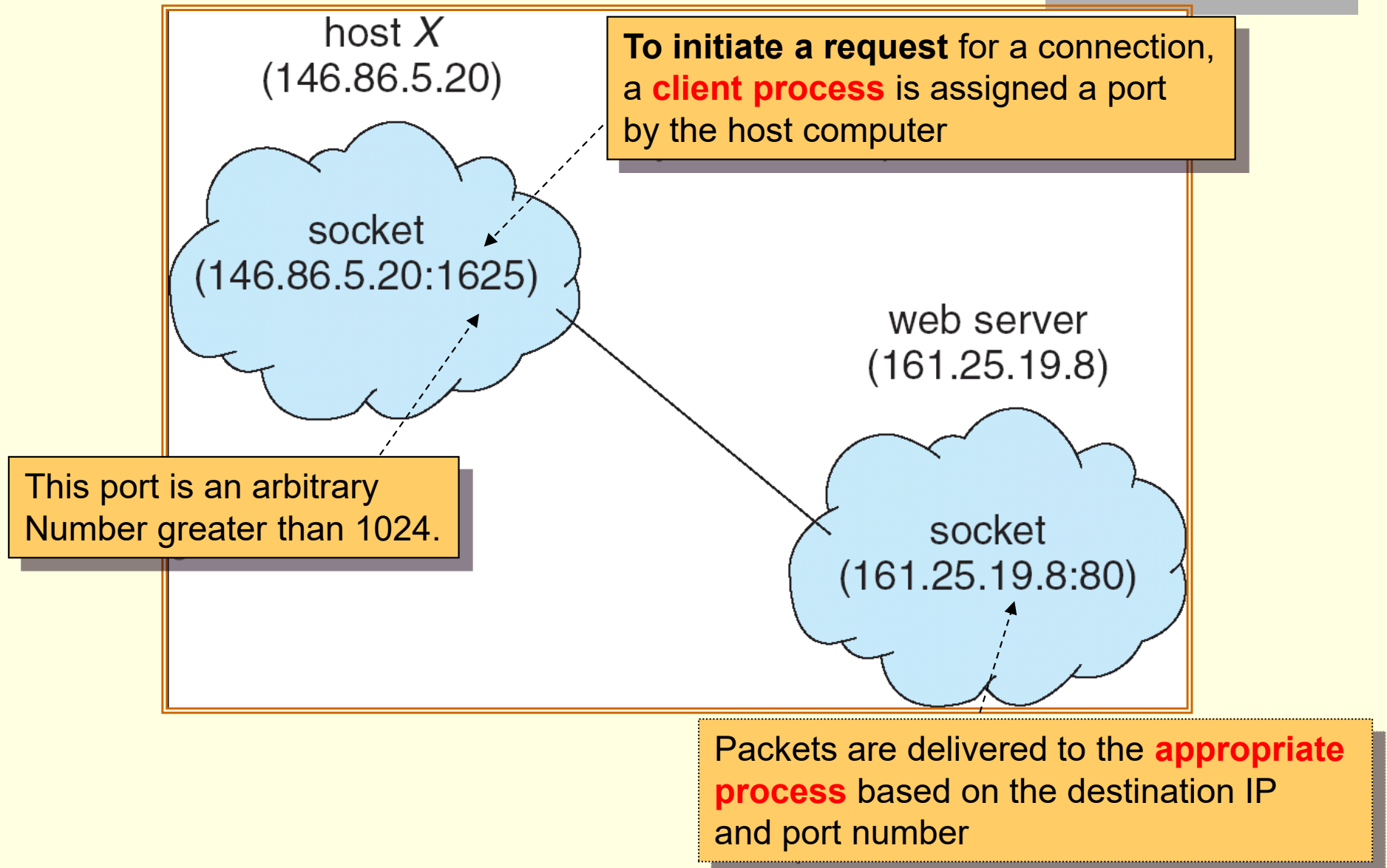
- Whether communication is direct or indirect, messages exchanged by communicating processes reside in a temporary **queue**.
- Queue can be:
 - **Zero capacity**: can not have any messages waiting in it.
 - The sender must block until the recipient receives the message.
 - **Bounded capacity**: the queue has finite length n .
 - If the queue is full, the sender must block until space is available in the queue.
 - **Unbounded capacity**: the sender never blocks.

Communication in Client-Server Systems

Sockets (1/2)

- A **socket** is defined as an endpoint for communication.
- A socket is identified by an **IP address** and a **port number**.
 - The socket **161.25.19.8:1625** refers to port **1625** on host **161.25.19.8**.
- A pair of processes communicating over a network employ a pair of sockets – one for each process.
 - The server waits for incoming client requests by listening to a specified port.
 - Once a request is received, the server accepts (constructs) a connection from the client socket to complete the connection.
- Why port?
 - A way to communicate is to know the process ID of the hosts.
 - However, IDs change frequently.
 - Each port represents a network services.
 - All ports below 1024 are considered **well known**.
 - http – 80, telnet – 23, ftp – 21 ...

Sockets (2/2)



Java Sockets Example (1/4)

- Java provides three different types of sockets:
 - **Connection-oriented** (TCP) sockets – implemented with the `Socket` class.
 - **Connectionless** (UDP) sockets – implemented with the `DatagramSocket` class.
 - **Multicast sockets** – implemented with `MulticastSocket` class.

- Example Server:
 - Use connection-oriented TCP sockets.
 - Listen to port 6013.
 - When a client request is received, the server returns the data and time to the client.

Java Sockets Example (2/4)

DateServer.java

```
import java.net.*;
import java.io.*;

public class DateServer
{
    public static void main(String[] args) {
        try {
            ServerSocket sock = new ServerSocket(6013);

            // now listen for connections
            while (true) {
                Socket client = sock.accept();
                // we have a connection

                PrintWriter pout = new PrintWriter(client.getOutputStream(), true);
                // write the Date to the socket
                pout.println(new java.util.Date().toString());

                // close the socket and resume listening for more connections
                client.close();
            }
        } catch (IOException ioe) {
            System.err.println(ioe);
        }
    }
}
```

Creates a server socket object, bound to the specified port.

Listens for (and **block till**) a connection to be made to this socket and accepts it.

Return a socket that the server can use to communicate with the client.

The server closes the socket to the client and resumes listening for more requests.

<http://java.sun.com/j2se/1.4.2/docs/api/java/net/ServerSocket.html>

Java Sockets Example (3/4)

```
import java.net.*;
import java.io.*;
```

DateClient.java

```
public class DateClient
{
```

```
    public static void main(String[] args) {
```

```
        try {
```

```
            // this could be changed to an IP name
```

```
            // or address other than the localhost
```

```
            Socket sock = new Socket("127.0.0.1", 6013);
```

```
            InputStream in = sock.getInputStream();
```

```
            BufferedReader bin = new BufferedReader(new
                                                    InputStreamReader(in));
```

```
            String line;
```

```
            while( (line = bin.readLine()) != null)
```

```
                System.out.println(line);
```

```
            sock.close();
```

```
        } catch (IOException ioe) {
```

```
            System.err.println(ioe);
```

```
        }
```

```
    }
```

```
}
```

Create a socket and request a connection with the server on port 6013

Java Sockets Example (4/4)

```
C:\ 命令提示字元 - java DateServer
D:\upload>javac DateServer.java
D:\upload>java DateServer
```

```
C:\ 命令提示字元
D:\upload>javac DateClient.java
D:\upload>java DateClient
Mon Mar 10 11:23:34 CST 2008
D:\upload>_
```

- The IP address 127.0.0.1 is a special IP address known as the *loopback*.
 - **it is referring to itself!!**
 - The example runs the client and server on the same host to communicate using the TCP/IP protocol.

Pipes (1/6)

- Pipes were one of the first IPC mechanisms in early UNIX systems.
- Two common types of pipes used on both UNIX and Windows systems:
 - **Ordinary pipes** and **named pipes**.

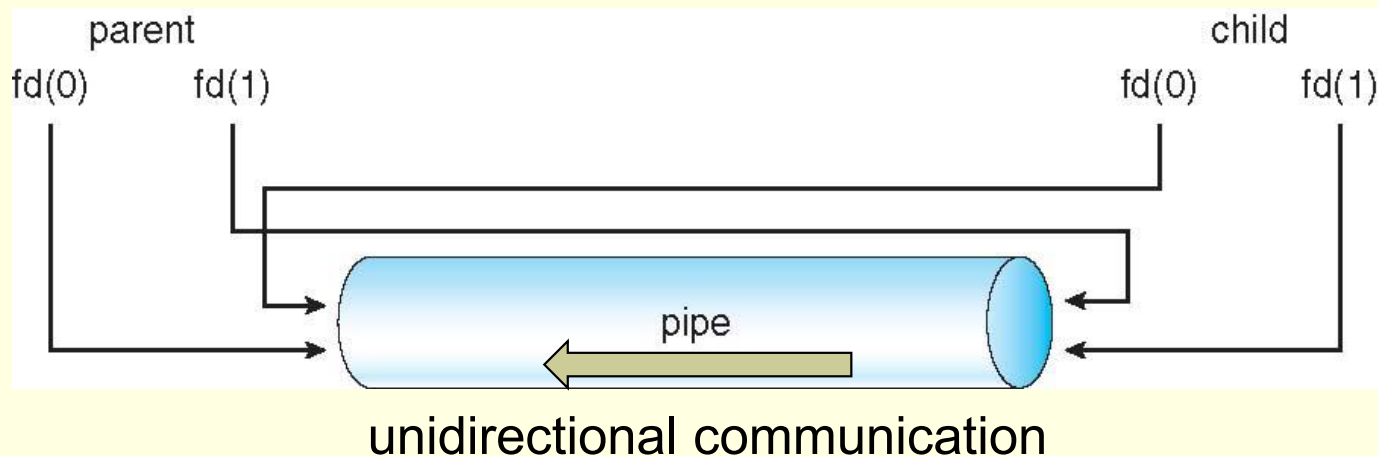
Pipes (2/6)

- Ordinary pipes:
 - Are **unidirectional**.
 - Allow two processes to communicate in standard producer-consumer fashion.
 - The producer writes to one end of the pipe (the **write-end**).
 - The consumer reads from the other end (the **read-end**).
 - On UNIX systems, ordinary pipes are constructed using the **pipe()** system call (POSIX).
pipe(int fd[])
 - The function creates a pipe that is accessed through the `int fd[]` file descriptor.
 - `fd[0]` is the read end.
 - `fd[1]` is the write end.
 - UNIX treats a pipe as a file, so pipes can be accessed using `read()` and `write()` system calls.

Pipes (3/6)

■ Ordinary pipes:

- An ordinary pipe **cannot** be accessed from outside the process that created it!!
- Generally, a parent creates a pipe and uses it to communicate with a child that it creates via `fork()`.
- A forked child inherits open files from its parent.
 - Pipe is a special type of file, so the child can access the pipe too.



Pipes (4/6)

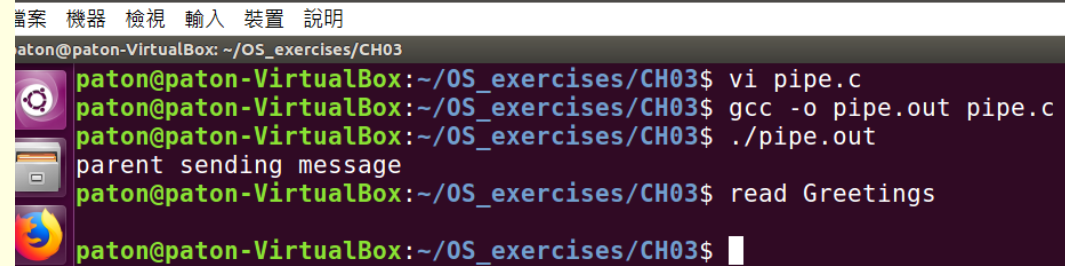
```
#include <sys/types.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>

#define BUFFER_SIZE 25
#define READ_END 0
#define WRITE_END 1

int main(void)
{
    char write_msg[BUFFER_SIZE] = "Greetings";
    char read_msg[BUFFER_SIZE];
    int fd[2];
    pid_t pid;

    // create a pipe
    if(pipe(fd) == -1) {
        fprintf(stderr, "Pipe failed");
        return 1;
    }
```

Pipes (5/6)



```
paton@paton-VirtualBox: ~/OS_exercises/CH03
paton@paton-VirtualBox:~/OS_exercises/CH03$ vi pipe.c
paton@paton-VirtualBox:~/OS_exercises/CH03$ gcc -o pipe.out pipe.c
paton@paton-VirtualBox:~/OS_exercises/CH03$ ./pipe.out
parent sending message
paton@paton-VirtualBox:~/OS_exercises/CH03$ read Greetings
paton@paton-VirtualBox:~/OS_exercises/CH03$
```

```
pid = fork ();

if (pid < 0) {
    fprintf(stderr, "Fork failed");
    return 1;
}

if(pid > 0) { // parent process
    close(fd[READ_END]);
    printf("parent sending message\n");
    write(fd[WRITE_END], write_msg, strlen(write_msg)+1);
    close(fd[WRITE_END]);
} else { // child process
    close(fd[WRITE_END]);
    read(fd[READ_END], read_msg, BUFFER_SIZE);
    printf("read %s\n", read_msg);
    close(fd[READ_END]);
}

return 0;
}
```

It is better to close unused ends of the pipe

Pipes (6/6)

- Ordinary pipes require a parent-child relationship between the communicating processes (on both UNIX and Windows).
 - What if we want communication with no parent-child relation?
- Both UNIX and Windows systems support **named pipes**.
 - Referred to as FIFOs in UNIX systems.
 - Once created, they appear as a files in the file systems.
 - Continue to exist until it is explicitly deleted from the file system.
 - You can create named pipes using `mkfifo()` system call (POSIX).

End of Chapter 3

Homework 1: